GameShop Interview Preparation Guide

Java • Spring Boot • Spring Security • JDBC • MySQL • React • OOP • Collections • Exceptions

A practical question-and-answer guide based on the GameShop project and the resume bullets we prepared.

How to use this guide: Read the question, answer it aloud in your own words, then compare your response with the sample. Do not memorize claims that you have not verified in your source code.

Project overview

GameShop is a full-stack gaming-café booking application. The React frontend lets customers request sessions and gives administrators a dashboard to manage bookings and gaming stations. The Spring Boot backend applies booking rules and communicates with a MySQL database.

Known project details used here

- Backend: Java 21 and Spring Boot; frontend: React/Vite; database: MySQL.
- The backend has booking and gaming-station controllers, services, repositories, request DTOs, and a global exception handler.
- The booking API includes GET /api/bookings, POST /api/bookings, and PUT endpoints to confirm or cancel a booking.
- Booking creation validates input and opening hours (10:00–23:00); new bookings are PENDING and have no assigned station.
- When confirming a booking, the service locks/reads stations for update, chooses an available operational station, and assigns it.
- Cancelling a booking changes its status to CANCELLED and clears its assigned station.
- Security configuration uses session-based authentication, admin role checks, and CookieCsrfTokenRepository.

Important accuracy note: the guide explains the implementation described in our project notes. Check your current code before claiming a particular annotation, collection type, concurrency technique, or test result.

1. Project overview and architecture

Q. Tell me about your GameShop project.

Sample answer: GameShop is a full-stack gaming-café booking system built with React, Spring Boot, and MySQL. Customers can request a gaming session, while an administrator can manage bookings and gaming stations. The backend validates booking requests, checks availability, and assigns an operational station when a booking is confirmed.

Q. Why did you choose this project?

Sample answer: I wanted to build something more realistic than a basic CRUD application. It has related data, booking rules, admin access control, and station assignment. It gave me a way to practise Java backend development and connect a React frontend to a database-backed API.

Q. Explain the architecture from frontend to database.

Sample answer: The user interacts with the React frontend. React sends an HTTP request to a Spring Boot controller. The controller accepts the request and calls the service layer, where the business rules are applied. The repository uses JdbcTemplate and SQL to read or update MySQL. The backend returns an HTTP response to React.

Q. Why separate controller, service, and repository layers?

Sample answer: The controller handles HTTP requests and responses, the service owns business rules, and the repository handles database access. Separating responsibilities makes the code easier to understand and change. For example, booking availability rules belong in the service rather than being mixed into the controller.

Q. What happens when a customer creates a booking?

Sample answer: The frontend sends the customer's details, start time, and duration. The backend validates the request and opening hours, checks station availability, sets the initial status to PENDING with no assigned station, and saves the booking. The database generates the booking ID, which the backend returns.

Q. What happens when an admin confirms a booking?

Sample answer: The service checks that the booking is still pending, finds an operational station that is available for the requested time, and assigns it. If no station is available or the booking is not in a confirmable state, the service rejects the operation rather than assigning an unsuitable station.

Q. What happens when a booking is cancelled?

Sample answer: The booking status is updated to CANCELLED and its assigned station is cleared. That means the cancelled booking should no longer reserve the station. The application should still prevent invalid state changes according to its business rules.

Q. What would you improve next?

Sample answer: I would add booking search by customer name or booking ID and a clear session lifecycle such as upcoming, in progress, and completed. I would also finish deployment configuration, verify security settings, and add targeted tests for booking conflicts and station assignment.

2. REST APIs and HTTP

Q. What is a REST API?

Sample answer: A REST API lets separate parts of an application communicate using HTTP endpoints. In GameShop, React calls Spring Boot endpoints to create bookings, retrieve stations, and let an authorized admin manage bookings.

Q. Which HTTP methods does GameShop use?

Sample answer: GET retrieves data, POST creates a booking, and PUT is used for booking actions such as confirming or cancelling. The exact method and endpoint depend on the operation.

Q. What is the difference between a path variable and a request body?

Sample answer: A path variable is part of the URL, such as the booking ID in /api/bookings/12/confirm. A request body contains data sent with the request, such as the customer's name, start time, and duration when creating a booking.

Q. What is a DTO, and why use CreateBookingRequest?

Sample answer: A DTO, or Data Transfer Object, represents the data accepted from or returned to an API client. CreateBookingRequest defines the input needed to request a booking, instead of exposing the database model as the request contract.

Q. What do HTTP 400 and 409 mean in this project?

Sample answer: HTTP 400 Bad Request is used for invalid input, such as a booking request that fails validation. HTTP 409 Conflict is used when a request conflicts with the current application state, such as a booking operation that cannot be completed.

Q. What is the difference between frontend validation and backend validation?

Sample answer: Frontend validation helps users correct mistakes quickly, but it can be bypassed. Backend validation is essential because the server must enforce the rules regardless of how the request was sent.

3. Spring Boot and annotations

Q. What is Spring Boot?

Sample answer: Spring Boot simplifies building and running Spring applications by providing auto-configuration, dependency management, and an embedded web server. In GameShop it runs the Java backend and exposes the API.

Q. What does @SpringBootApplication do?

Sample answer: It is the main Spring Boot annotation. It combines configuration, auto-configuration, and component scanning so Spring can configure the application and discover its components.

Q. What does @RestController do?

Sample answer: It marks a class as a web controller whose methods return response data, commonly JSON, instead of rendering a traditional server-side view.

Q. What is @Service used for?

Sample answer: It marks a service component that contains application business logic. In GameShop, the booking service handles validation, status changes, availability, and station assignment.

Q. What is @Repository used for?

Sample answer: It identifies a data-access component. A repository is responsible for database operations, keeping SQL and persistence work separate from business rules.

Q. What do @Configuration and @Bean do in SecurityConfig?

Sample answer: @Configuration marks a class that defines Spring configuration. @Bean tells Spring to manage the object returned by a method. In the security configuration, the bean method creates the SecurityFilterChain.

Q. What is dependency injection?

Sample answer: Dependency injection means a class receives the objects it needs from the framework instead of creating every dependency itself. It reduces tight coupling and makes components easier to manage and test.

Q. What does @Valid do with request validation?

Sample answer: When used on a request DTO, @Valid asks the validation framework to check its configured constraints, such as a required name or a minimum duration. The backend must then handle invalid requests appropriately.

Q. What is a Spring bean?

Sample answer: A Spring bean is an object managed by the Spring application context. Spring can create it, configure its dependencies, and provide it to other components.

4. JDBC, JdbcTemplate, and MySQL

Q. What is JDBC?

Sample answer: JDBC stands for Java Database Connectivity. It is a Java API used to communicate with relational databases.

Q. What is JdbcTemplate?

Sample answer: JdbcTemplate is a Spring utility built on JDBC. It simplifies executing SQL statements and handling common JDBC tasks, such as resource cleanup and translating database errors.

Q. Why use JdbcTemplate instead of connecting React directly to MySQL?

Sample answer: The browser should not connect directly to the database because that would expose database access details and bypass backend business rules. React calls the backend API, and the backend securely accesses MySQL.

Q. How is a new booking ID generated?

Sample answer: The booking insert omits the auto-increment booking_id. MySQL generates the ID, and the repository retrieves the generated key and sets it on the Booking object.

Q. Why use parameterized SQL with placeholders?

Sample answer: Placeholders keep user-provided values separate from SQL text, helping prevent SQL injection and ensuring values are handled as parameters instead of executable SQL.

Q. What is a foreign key in this project?

Sample answer: The assigned_station_id in a booking can reference a gaming station. A foreign key helps preserve referential integrity so a booking cannot refer to a station row that does not exist.

Q. How do you keep credentials out of application.properties?

Sample answer: The configuration uses environment-variable placeholders for the database URL, username, and password. Local development variables are configured in the IDE, while deployment variables must be configured in the hosting platform. Real secrets should not be committed to Git.

Q. What is the difference between JDBC and JPA/Hibernate?

Sample answer: JDBC works with SQL and database operations more directly. JPA is a persistence specification for mapping Java objects to relational data, and Hibernate is a common JPA implementation. GameShop's described repository uses JdbcTemplate rather than relying on JPA entities for those operations.

5. Spring Security, sessions, and CSRF

Q. What is authentication?

Sample answer: Authentication checks who the user is, for example by verifying login credentials.

Q. What is authorization?

Sample answer: Authorization checks what an authenticated user is allowed to do. GameShop requires the ADMIN role for protected booking-management and station-management endpoints.

Q. What does hasRole("ADMIN") mean?

Sample answer: It restricts the matched endpoint to an authenticated user with the ADMIN role. Spring Security commonly represents that role internally as ROLE_ADMIN.

Q. What is session-based authentication?

Sample answer: After successful login, the server maintains an authenticated session and the browser sends the session cookie with later requests. The server uses that session to identify the logged-in user.

Q. What is CSRF?

Sample answer: Cross-Site Request Forgery is an attack where a browser is tricked into sending an unwanted request to a site where the user is already authenticated. CSRF protection requires a token on protected requests to help defend against this.

Q. How does GameShop handle the CSRF token?

Sample answer: The security configuration uses CookieCsrfTokenRepository. The frontend obtains the CSRF token and includes it with state-changing requests, such as confirming or cancelling a booking. I should be able to point to the actual frontend request code when explaining the implementation.

Q. Does CSRF protection replace authentication or authorization?

Sample answer: No. Authentication identifies the user, authorization checks permissions, and CSRF protection helps prevent forged requests that misuse the user's browser session.

Q. Why shouldn't admin credentials be hardcoded?

Sample answer: Hardcoded credentials can be exposed through source code and Git history. Environment variables or deployment secret settings keep credentials outside the committed application source. Passwords should be rotated if they are ever exposed.

6. OOP concepts you can explain

Q. How does OOP relate to GameShop?

Sample answer: The code is organized into classes with specific responsibilities. Booking and GamingStation represent domain data, while controllers, services, and repositories handle API, business, and persistence responsibilities. This structure makes the application easier to reason about.

Q. What is encapsulation?

Sample answer: Encapsulation means keeping an object's state and the operations on that state together, while controlling access to its internal data. For example, a Booking model can expose fields through methods rather than allowing unrelated parts of the program to manipulate internal state arbitrarily.

Q. What is abstraction?

Sample answer: Abstraction means exposing the important operations while hiding implementation details. A controller can call a booking service method without needing to know the SQL used by the repository.

Q. What is inheritance?

Sample answer: Inheritance lets a class derive fields or behaviour from another class. Do not claim a custom inheritance hierarchy in GameShop unless you can identify it in your code. You can explain inheritance as a Java concept separately.

Q. What is polymorphism?

Sample answer: Polymorphism allows the same interface or parent type to be used with different implementations. Only give a project-specific example if your code actually uses an interface or overridden methods polymorphically.

Q. Why split the application into model, DTO, controller, service, and repository classes?

Sample answer: Each type has a distinct responsibility: models represent domain data, DTOs define API input, controllers handle HTTP, services enforce business rules, and repositories access the database. This separation supports maintainability and reduces mixing unrelated concerns.

Q. Is a class the same as an object?

Sample answer: A class is a blueprint that defines data and behaviour. An object is an instance created from that class, such as a Booking object representing one customer's booking.

7. Collections Framework

Be precise here: the project notes confirm the repository and service structure, but do not establish every collection type used in the final source. Inspect your code before claiming a specific collection.

Q. What is the Java Collections Framework?

Sample answer: It is a set of interfaces and classes for storing and working with groups of objects, including List, Set, Queue, ArrayList, LinkedList, and HashMap.

Q. What is the difference between List, Set, and Map?

Sample answer: A List is ordered and can contain duplicates. A Set stores unique elements. A Map stores key-value pairs, such as mapping an ID to an object.

Q. What is an ArrayList?

Sample answer: ArrayList is a resizable array implementation of List. It is useful for indexed access and iterating over a sequence of objects.

Q. What is a HashMap?

Sample answer: HashMap stores key-value pairs and typically provides efficient lookup by key. It does not guarantee iteration order.

Q. How might collections appear in GameShop?

Sample answer: A service might receive a List of gaming stations from the repository and iterate through it to find an operational station. Only say this is exactly how your implementation works if your current source confirms the return type and code.

Q. How should I answer if I did not use a particular collection directly?

Sample answer: I would say that I understand the concept, but I would not claim that I used it in this project unless I can identify where. Then I would explain a collection I actually used in my code.

8. Exceptions, validation, and error handling

Q. What is an exception?

Sample answer: An exception is an event that interrupts the normal flow of a program. Java lets code throw and catch exceptions so failures can be handled in a controlled way.

Q. How does GameShop handle invalid booking requests?

Sample answer: The request DTO uses validation constraints, and the service also checks business rules such as opening hours and booking availability. Invalid input should be rejected rather than stored as a valid booking.

Q. What does GlobalExceptionHandler do?

Sample answer: It centralizes handling for selected exceptions and converts them into HTTP responses. In the described implementation, IllegalArgumentException maps to HTTP 400 and IllegalStateException maps to HTTP 409.

Q. Why return HTTP 409 for a booking conflict?

Sample answer: The request may be syntactically valid, but the requested operation conflicts with the current state, for example when a booking is no longer pending or a suitable station is unavailable.

Q. What is the difference between checked and unchecked exceptions?

Sample answer: Checked exceptions must be caught or declared by the method. Unchecked exceptions are subclasses of RuntimeException and do not have that compile-time requirement. IllegalArgumentException and IllegalStateException are unchecked exceptions.

Q. What is the difference between throw and throws?

Sample answer: throw is used to actually throw an exception object. throws declares in a method signature that the method may pass certain exceptions to its caller.

Q. What real technical problems did you face?

Sample answer: A genuine issue during local setup was a port conflict because port 8080 was already in use. There was also a database connection credential issue during a test run. Explain what the logs showed and how you investigated them; do not claim these were application logic bugs.

Q. How would you handle an unexpected database failure?

Sample answer: I would inspect the exception and logs, avoid exposing sensitive internal details to the client, and return an appropriate server error. The exact implementation should match the application's exception-handling configuration.

9. Multithreading and concurrency

Important: the project notes do not establish that GameShop itself implements a custom multithreading module. Do not claim that it does. The multithreaded GPIO motor-control module belongs to your separate Raspberry Pi RC-car project, according to your resume.

Q. Did you implement multithreading in GameShop?

Sample answer: I would not describe GameShop as having a custom multithreading feature unless I can identify it in the source. The web server and database libraries handle concurrent requests internally, but that is different from me implementing a custom thread-management feature.

Q. What is multithreading in Java?

Sample answer: Multithreading allows multiple threads of execution within a process. It can help perform independent tasks concurrently, but shared mutable data must be handled carefully.

Q. What is a race condition?

Sample answer: A race condition happens when the result depends on the timing or interleaving of concurrent operations on shared state. In a booking system, two requests could attempt to assign the same station for overlapping time slots.

Q. How does GameShop approach station assignment concurrency?

Sample answer: The described service uses a repository method that retrieves stations for update and applies station assignment logic during confirmation. The intention is to coordinate assignment against the database state. I would explain the exact transaction boundaries and locking behaviour only after checking the current code, because the lock's effectiveness depends on the transaction and database configuration.

Q. Does synchronized automatically solve database booking conflicts?

Sample answer: No. synchronized coordinates threads within one JVM, but it does not coordinate separate application instances. Database transactions, appropriate locking, and integrity rules are often needed for conflicts across requests and server instances.

Q. How is multithreading different from asynchronous programming?

Sample answer: Multithreading uses multiple threads of execution. Asynchronous programming lets work proceed without blocking the caller while waiting for an operation; it may or may not use multiple threads, depending on the implementation.

10. Booking rules and scenario questions

Q. How do you prevent a booking outside opening hours?

Sample answer: The service validates the requested start and end times against the configured opening hours, which are 10:00 to 23:00 in the current project notes. The request should be rejected if it falls outside the allowed window.

Q. Why are new bookings PENDING?

Sample answer: Creating a request and confirming a station assignment are separate business steps. PENDING means the booking has been requested but not yet confirmed and assigned.

Q. Why is the station assigned during confirmation rather than creation?

Sample answer: It lets the system validate availability at the time the admin confirms the booking and assign an operational station then. The service must still recheck availability so that an earlier availability result is not treated as a guarantee.

Q. What should happen if all stations are unavailable?

Sample answer: The booking should not be assigned to an unavailable station. The service should reject confirmation with a clear conflict response, allowing the admin to choose an appropriate next action.

Q. What should happen when a confirmed booking is cancelled?

Sample answer: The status should change to CANCELLED and the assigned station reference should be cleared, so the cancelled booking no longer reserves that station.

Q. What is one improvement you would add?

Sample answer: I would add search by customer name or booking ID and a session lifecycle view for upcoming, in-progress, and completed sessions. I would also add tests for overlapping booking requests and verify the behaviour under concurrent confirmations.

11. Questions you should ask yourself before an interview

- Can I trace one booking request from React to the controller, service, repository, and database?
- Can I explain every annotation I mention and point to where it appears in the source?
- Can I explain the SQL used to insert and update a booking?
- Can I explain how the generated booking ID is retrieved?
- Can I distinguish authentication, authorization, and CSRF protection?
- Can I describe the actual collection types present in the code?
- Can I explain the GlobalExceptionHandler mappings and why 400 differs from 409?
- Can I explain the station assignment and locking logic without overstating its concurrency guarantees?
- Can I honestly distinguish what I implemented from what I learned with help?
- Can I run the application and demonstrate the features I list on my resume?

Final interview tip

A strong answer is specific, honest, and understandable. If you do not know something, say so, then explain what you do know and how you would investigate. It is better to accurately describe one real implementation than to use advanced terms you cannot defend.